

```
In [2]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

In [3]: df=pd.read_csv("C:\\Users\\Dell\\Desktop\\Project 1\\Ecommerce retail\\ecommerce_retail_transactions_raw.csv", encoding='latin1')
df.head()
```

Out[3]:

	Order_ID	Customer_ID	Order_Date	Product_Category	Product_Name	Quantity	Unit_Price_USD	Discount_Percent	Payment_Method	Shipping_City	Country	Order_Status	Custo
0	ORD011416	CUST00003	2026-04-03	Books	Self-Help Book	3	9.16	5.0	Net Banking	London	UK	Delivered	
1	ORD000974	CUST00269	10-15-2024	Books	Fiction Novel	1	34.24	5.0	NetBanking	Toronto	CA	Shipped	
2	ORD000617	CUST02075	17 Jun 2024	Home & Kitchen	Vacuum Cleaner	2	111.95	15.0	DEBIT CARD	Brisbane	Australia	Shipped	
3	ORD006437	CUST02109	16/10/2025	Beauty	Hair Dryer	1	31.84	0.0	Cash on Delivery	Birmingham	U.K.	Delivered	
4	ORD000740	CUST00349	06 Apr 2026	Home & Kitchen	Non-Stick Pan	2	72.00	NaN	Credit_Card	Montreal	CA	Delivered	

Cleaning

```
In [4]: # To replace underscores (.) with space in all column names
df.columns=df.columns.str.replace("_", " ")
df.columns
```

Out[4]:

Index(['Order ID', 'Customer ID', 'Order Date', 'Product Category',
 'Product Name', 'Quantity', 'Unit Price USD', 'Discount Percent',
 'Payment Method', 'Shipping City', 'Country', 'Order Status',
 'Customer Rating'],
 dtype='object')

```
In [5]: df.dtypes
```

Out[5]:

Order ID object
Customer ID object
Order Date object
Product Category object
Product Name object
Quantity int64
Unit Price USD float64
Discount Percent float64
Payment Method object
Shipping City object
Country object
Order Status object
Customer Rating float64
dtype: object

```
In [6]: df.head()
```

Out[6]:

	Order ID	Customer ID	Order Date	Product Category	Product Name	Quantity	Unit Price USD	Discount Percent	Payment Method	Shipping City	Country	Order Status	Customer Rating
0	ORD011416	CUST00003	2026-04-03	Books	Self-Help Book	3	9.16	5.0	Net Banking	London	UK	Delivered	5.0
1	ORD000974	CUST00269	10-15-2024	Books	Fiction Novel	1	34.24	5.0	NetBanking	Toronto	CA	Shipped	NaN
2	ORD000617	CUST02075	17 Jun 2024	Home & Kitchen	Vacuum Cleaner	2	111.95	15.0	DEBIT CARD	Brisbane	Australia	Shipped	NaN
3	ORD006437	CUST02109	16/10/2025	Beauty	Hair Dryer	1	31.84	0.0	Cash on Delivery	Birmingham	U.K.	Delivered	3.0
4	ORD000740	CUST00349	06 Apr 2026	Home & Kitchen	Non-Stick Pan	2	72.00	NaN	Credit_Card	Montreal	CA	Delivered	NaN

```
In [7]: # To check NaN Value in Each Columns Wise before
df.isna().sum()
```

Out[7]:

Order ID 0
Customer ID 0
Order Date 0
Product Category 0
Product Name 0
Quantity 0
Unit Price USD 0
Discount Percent 735
Payment Method 0
Shipping City 502
Country 0
Order Status 0
Customer Rating 6150
dtype: int64

```
In [8]: # To Check which one will be better to replace with Nan in Customer Rating
print(df["Customer Rating"].mean())
print(df["Customer Rating"].median())
```

3.896185737976783
4.0

```
In [9]: # Filling Discount Percent With 0 where value is NaN
df["Discount Percent"]=df["Discount Percent"].fillna(0)
# Filling Shipping City With "Unknown" where value is NaN
df["Shipping City"]=df["Shipping City"].fillna("Unknown")
# Filling Customer Rating with Median where Value is Nan
df["Customer Rating"]=df["Customer Rating"].fillna(df["Customer Rating"].median())
```

```
In [10]: # To check NaN Value in Each Columns Wise after
df.isna().sum()
```

```
Out[10]: Order ID      0
Customer ID    0
Order Date     0
Product Category 0
Product Name   0
Quantity       0
Unit Price USD 0
Discount Percent 0
Payment Method 0
Shipping City   0
Country        0
Order Status    0
Customer Rating 0
dtype: int64
```

```
In [11]: # To check duplicate value on the basis of orde id before
df["Order ID"].duplicated().sum()
```

Out[11]: np.int64(180)

```
In [12]: # To delete duplicate value on the basis of order id
df=df.drop_duplicates(subset=["Order ID"], keep="first")
# To check duplicate value on the basis of orde id before
df["Order ID"].duplicated().sum()
```

Out[12]: np.int64(0)

```
In [13]: # To replace float value into percentage(decimal) value in discount
df["Discount Percent"]=df["Discount Percent"]/100
# Add Sales Amount column by multiply unit price and qty
df["Sales Amount"]=df["Unit Price USD"]*df["Quantity"]
# Add Total Amount column after discount
df["Total Amount"]=df["Sales Amount"]*(1-df["Discount Percent"])
```

```
In [14]: df.head()
```

Out[14]:

	Order ID	Customer ID	Order Date	Product Category	Product Name	Quantity	Unit Price USD	Discount Percent	Payment Method	Shipping City	Country	Order Status	Customer Rating	Sales Amount	Total Amount
0	ORD011416	CUST00003	2026-04-03	Books	Self-Help Book	3	9.16	0.05	Net Banking	London	UK	Delivered	5.0	27.48	26.106
1	ORD000974	CUST00269	10-15-2024	Books	Fiction Novel	1	34.24	0.05	NetBanking	Toronto	CA	Shipped	4.0	34.24	32.528
2	ORD000617	CUST02075	17 Jun 2024	Home & Kitchen	Vacuum Cleaner	2	111.95	0.15	DEBIT CARD	Brisbane	Australia	Shipped	4.0	223.90	190.315
3	ORD006437	CUST02109	16/10/2025	Beauty	Hair Dryer	1	31.84	0.00	Cash on Delivery	Birmingham	U.K.	Delivered	3.0	31.84	31.840
4	ORD000740	CUST00349	06 Apr 2026	Home & Kitchen	Non-Stick Pan	2	72.00	0.00	Credit_Card	Montreal	CA	Delivered	4.0	144.00	144.000

```
In [15]: # Change format of data into Proper Format (Title)
proper_column=[
    "Product Category",
    "Product Name",
    "Payment Method",
    "Shipping City",
    "Country",
    "Order Status"
]
for i in proper_column:
    df[i]=df[i].str.strip().str.title().str.replace("_", " ")
```

```
In [16]: df.head()
```

Out[16]:

	Order ID	Customer ID	Order Date	Product Category	Product Name	Quantity	Unit Price USD	Discount Percent	Payment Method	Shipping City	Country	Order Status	Customer Rating	Sales Amount	Total Amount
0	ORD011416	CUST00003	2026-04-03	Books	Self-Help Book	3	9.16	0.05	Net Banking	London	Uk	Delivered	5.0	27.48	26.106
1	ORD000974	CUST00269	10-15-2024	Books	Fiction Novel	1	34.24	0.05	Netbanking	Toronto	Ca	Shipped	4.0	34.24	32.528
2	ORD000617	CUST02075	17 Jun 2024	Home & Kitchen	Vacuum Cleaner	2	111.95	0.15	Debit Card	Brisbane	Australia	Shipped	4.0	223.90	190.315
3	ORD006437	CUST02109	16/10/2025	Beauty	Hair Dryer	1	31.84	0.00	Cash On Delivery	Birmingham	U.K.	Delivered	3.0	31.84	31.840
4	ORD000740	CUST00349	06 Apr 2026	Home & Kitchen	Non-Stick Pan	2	72.00	0.00	Credit Card	Montreal	Ca	Delivered	4.0	144.00	144.000

```
In [17]: # To Check data is repeated or not cause of different format before
df["Country"].value_counts()
```

Out[17]:

Country	
Uae	1170
Australia	1167
Germany	1160
India	1115
Canada	1096
Uk	885
Usa	727
De	587
In	575
U.A.E	572
Au	540
Ca	539
U.K.	444
United Kingdom	439
United States	341
Us	335
U.S.A	308

Name: count, dtype: int64

In [18]:

```
# Make one format for each country
country_format={
    "Uae":"UAE", "U.A.E":"UAE",
    "Au":"Australia",
    "De":"Germany",
    "In":"India",
    "Ca":"Canada",
    "Uk":"UK", "U.K.":"UK", "United Kingdom":"UK",
    "Usa":"USA", "United States":"USA", "Us":"USA", "U.S.A": "USA"}
df["Country"]=df["Country"].replace(country_format)
```

In [19]:

```
# To Check data is repeated or not cause of different format after
df["Country"].value_counts()
```

Out[19]:

Country	
UK	1768
Germany	1747
UAE	1742
USA	1711
Australia	1707
India	1690
Canada	1635

Name: count, dtype: int64

In [20]:

```
# To Check data is repeated or not cause of different format before
df["Payment Method"].value_counts()
```

Out[20]:

Payment Method	
Credit Card	2031
Debit Card	1910
Paypal	1390
Net Banking	1375
Upi	1293
Cod	1049
Cash On Delivery	1021
Netbanking	650
Pay Pal	643
U.P.I	638

Name: count, dtype: int64

In [21]:

```
# Make one format for each Payment Type
payment_format={
    "Pay Pal":"Paypal",
    "Net Banking":"Netbanking",
    "Upi":"UPI", "U.P.I":"UPI",
    "Cod":"Cash On Delivery"
}
df["Payment Method"]=df["Payment Method"].replace(payment_format)
```

In [22]:

```
# To Check data is repeated or not cause of different format after
df["Payment Method"].value_counts()
```

Out[22]:

Payment Method	
Cash On Delivery	2070
Paypal	2033
Credit Card	2031
Netbanking	2025
UPI	1931
Debit Card	1910

Name: count, dtype: int64

In [23]:

```
df.head()
```

Out[23]:

	Order ID	Customer ID	Order Date	Product Category	Product Name	Quantity	Unit Price USD	Discount Percent	Payment Method	Shipping City	Country	Order Status	Customer Rating	Sales Amount	Total Amount
0	ORD011416	CUST00003	2026-04-03	Books	Self-Help Book	3	9.16	0.05	Netbanking	London	UK	Delivered	5.0	27.48	26.106
1	ORD000974	CUST00269	10-15-2024	Books	Fiction Novel	1	34.24	0.05	Netbanking	Toronto	Canada	Shipped	4.0	34.24	32.528
2	ORD000617	CUST02075	17 Jun 2024	Home & Kitchen	Vacuum Cleaner	2	111.95	0.15	Debit Card	Brisbane	Australia	Shipped	4.0	223.90	190.315
3	ORD006437	CUST02109	16/10/2025	Beauty	Hair Dryer	1	31.84	0.00	Cash On Delivery	Birmingham	UK	Delivered	3.0	31.84	31.840
4	ORD000740	CUST00349	06 Apr 2026	Home & Kitchen	Non-Stick Pan	2	72.00	0.00	Credit Card	Montreal	Canada	Delivered	4.0	144.00	144.000

In [24]:

```
df.tail()
```

Out[24]:

	Order ID	Customer ID	Order Date	Product Category	Product Name	Quantity	Unit Price USD	Discount Percent	Payment Method	Shipping City	Country	Order Status	Customer Rating	Sales Amount	Total Amount
12175	ORD002515	CUST00071	2025-10-17	Home & Kitchen	Storage Organizer	1	106.25	0.00	Netbanking	Chicago	USA	Delivered	4.0	106.25	106.2500
12176	ORD011799	CUST02583	2024/01/22	Beauty	Sunscreen Spf50	1	24.76	0.15	Paypal	Mumbai	India	Delivered	5.0	24.76	21.0460
12177	ORD006638	CUST02113	02-28-2024	Grocery	Olive Oil 1L	1	29.95	0.10	UPI	Melbourne	Australia	Pending	4.0	29.95	26.9550
12178	ORD002576	CUST00298	2025-06-01	Beauty	Hair Dryer	1	41.41	0.15	Credit Card	Vancouver	Canada	Delivered	4.0	41.41	35.1985
12179	ORD007337	CUST00195	11-18-2024	Books	Self-Help Book	4	27.81	0.00	Netbanking	Mumbai	India	Delivered	4.0	111.24	111.2400

Creating KPI

In [25]:

```
# Business And Sales KPIs (Revenue Growth & Discount)
total_sales=df["Total Amount"].sum()
total_orders=df["Order ID"].nunique()
average_order_value=total_sales/total_orders
total_customers=df["Customer ID"].nunique()
average_customer_rating=df["Customer Rating"].sum()/len(df)
total_sold_quantity=df["Quantity"].sum()
total_discount=df["Sales Amount"].sum()-total_sales
average_discount=df["Discount Percent"].mean()*100
average_items_per_order=df["Quantity"].mean()
delivered_orders=df[df["Order Status"]=="Delivered"]["Order ID"].nunique()
delivery_rate=delivered_orders/total_orders*100

# Customer Performane KPIs (Retention & Value)
revenue_per_customer=df["Total Amount"].sum()/df["Customer ID"].nunique()
customer_order=df.groupby("Customer ID")["Order ID"].nunique()
repeat_customer=(customer_order > 1).sum()
repeat_rate=(repeat_customer/df["Customer ID"].nunique()*100)
```

In [26]:

```
print("=" * 40)
print("-----Business and Sales KPIs-----")
print("----(Revenue Growth & Discount)-----")
print("=" * 40)
print(f"Total Revenue: ${total_sales/1000000:,.2f}M")
print(f"Total Orders: {total_orders/1000:,.2f}K")
print(f"Average Order Value: ${average_order_value:,.2f}")
print(f"Total Sold Quantity: {total_sold_quantity/1000:,.2f}k")
print(f"Total Discoun Amount: ${total_discount/1000:,.2f}k")
print(f"Average Discount in Percentage: {average_discount:,.2f}%")
print(f"Average Items per Order: {average_items_per_order:,.0f}")
print(f"Delivered Order: {delivered_orders:,.0f}")
print(f"Delivery Rate: {delivery_rate:,.2f}%")
print()

print("=" * 40)
print("-----Customer Performane KPIs-----")
print("-----(Retention & Value)-----")
print("=" * 40)
print(f"Average Customer Rating: {average_customer_rating:,.2f}")
print(f"Total Cutomers: {total_customers/1000:,.3f}K")
print(f"Average Revenue Per Customer: ${revenue_per_customer:,.2f}")
print(f"Repeated Customer: {repeat_customer/1000:,.2f}K")
print(f"Repeat Rate: {repeat_rate:,.2f}%")

=====
----Business and Sales KPIs----
----(Revenue Growth & Discount)----
=====
Total Revenue: $1.39M
Total Orders: 12.00K
Average Order Value: $116.04
Total Sold Quantity: 24.23k
Total Discoun Amount: $102.07k
Average Discount in Percentage: 6.87%
Average Items per Order: 2
Delivered Order: 6,578
Delivery Rate: 54.82%

=====
----Customer Performane KPIs----
-----(Retention & Value)-----
=====
Average Customer Rating: 3.95
Total Cutomers: 3.374K
Average Revenue Per Customer: $412.70
Repeated Customer: 2.98K
Repeat Rate: 88.47%
```

Visualization

In [27]:

```
# Convert "Order Date" to datetime
df["Order Date"]=pd.to_datetime(df["Order Date"], format="mixed")
# Adding New Columns for check trend over year
df["Year"]=df["Order Date"].dt.year
df["Month"]=df["Order Date"].dt.month
```

In [28]:

```
# Reform as a str "DD-MM-YYYY"
df["Order Date"]=df["Order Date"].dt.strftime("%d-%m-%Y")
```

In [29]:

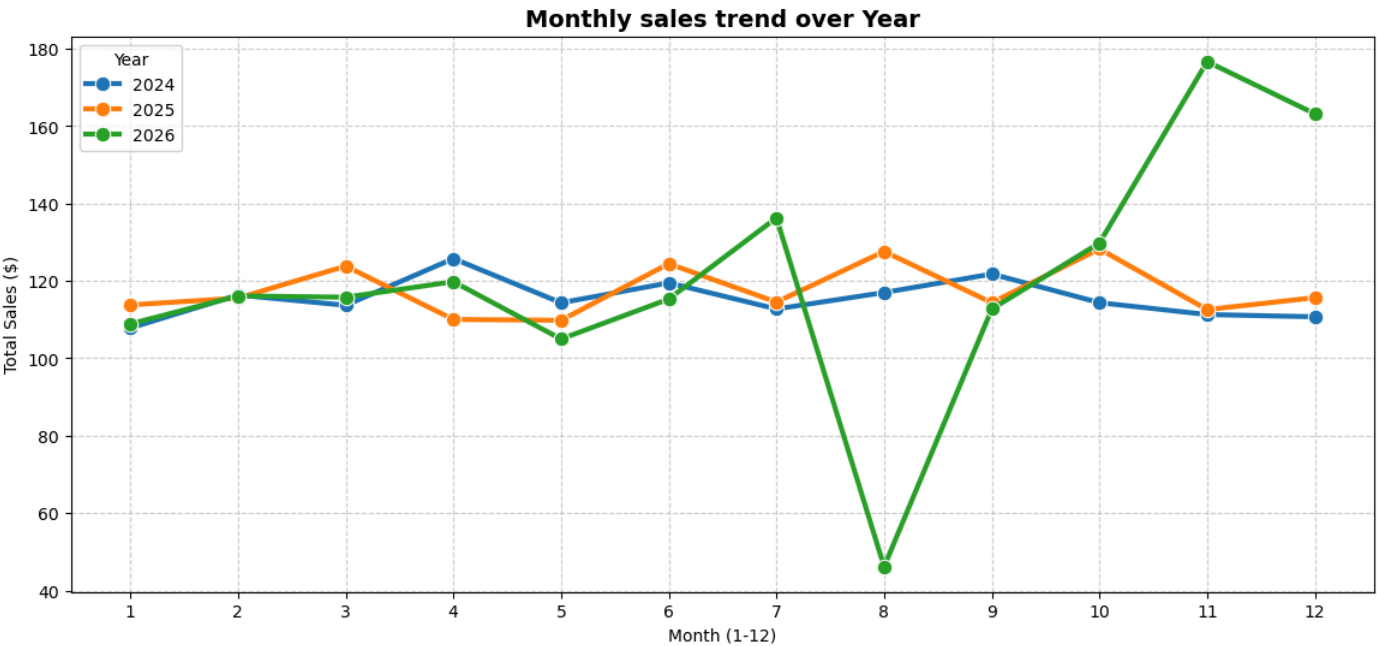
```
df.head()
```

Out[29]:

	Order ID	Customer ID	Order Date	Product Category	Product Name	Quantity	Unit Price USD	Discount Percent	Payment Method	Shipping City	Country	Order Status	Customer Rating	Sales Amount	Total Amount	Year	Month
0	ORD011416	CUST00003	03-04-2026	Books	Self-Help Book	3	9.16	0.05	Netbanking	London	UK	Delivered	5.0	27.48	26.106	2026	4
1	ORD000974	CUST00269	15-10-2024	Books	Fiction Novel	1	34.24	0.05	Netbanking	Toronto	Canada	Shipped	4.0	34.24	32.528	2024	10
2	ORD000617	CUST02075	17-06-2024	Home & Kitchen	Vacuum Cleaner	2	111.95	0.15	Debit Card	Brisbane	Australia	Shipped	4.0	223.90	190.315	2024	6
3	ORD006437	CUST02109	16-10-2025	Beauty	Hair Dryer	1	31.84	0.00	Cash On Delivery	Birmingham	UK	Delivered	3.0	31.84	31.840	2025	10
4	ORD000740	CUST00349	06-04-2026	Home & Kitchen	Non-Stick Pan	2	72.00	0.00	Credit Card	Montreal	Canada	Delivered	4.0	144.00	144.000	2026	4

Yearly & Monthly Sales Trend (Line Plot)

```
In [30]: plt.figure(figsize=(14,6))
sns.lineplot(
    x="Month",
    y="Total Amount",
    hue="Year",
    data=df,
    marker="o",
    palette="tab10",
    errorbar=None,
    linewidth=3,
    markersize=9
)
plt.title(
    "Monthly sales trend over Year",
    fontsize=14,
    fontweight="bold",
)
plt.xlabel("Month (1-12)")
plt.ylabel("Total Sales ($)")
plt.xticks(range(1,13))
plt.grid(True, linestyle="--", alpha=0.6)
plt.savefig('1_monthly_sales_trend.png', dpi=300, bbox_inches='tight')
plt.show()
```

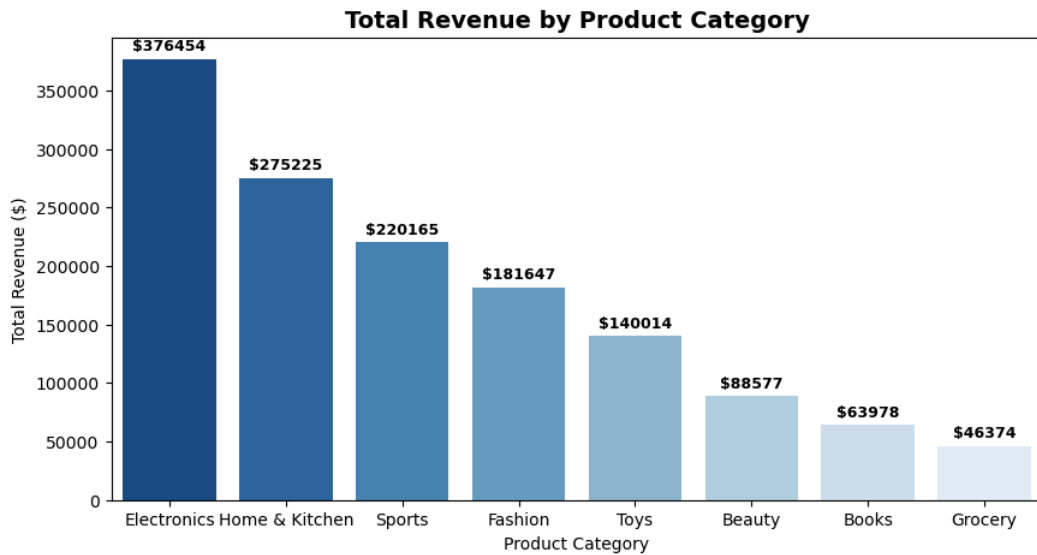


Top Product Categories by Revenue (Bar Chart)

```
In [31]: plt.figure(figsize=(10,5))
category_sales=df.groupby("Product Category")["Total Amount"].sum().sort_values(ascending=False).reset_index()

ax=sns.barplot(
    x="Product Category",
    y="Total Amount",
    data=category_sales,
    hue="Product Category",
    legend=False,
    palette="Blues_r"
)
for i in ax.containers:
    ax.bar_label(
        i,
        fmt="%.0f",
        padding=3,
        fontweight="bold",
        fontsize=9
    )
plt.title(
    "Total Revenue by Product Category",
    fontsize=14,
    fontweight="bold"
)
plt.xlabel("Product Category")
```

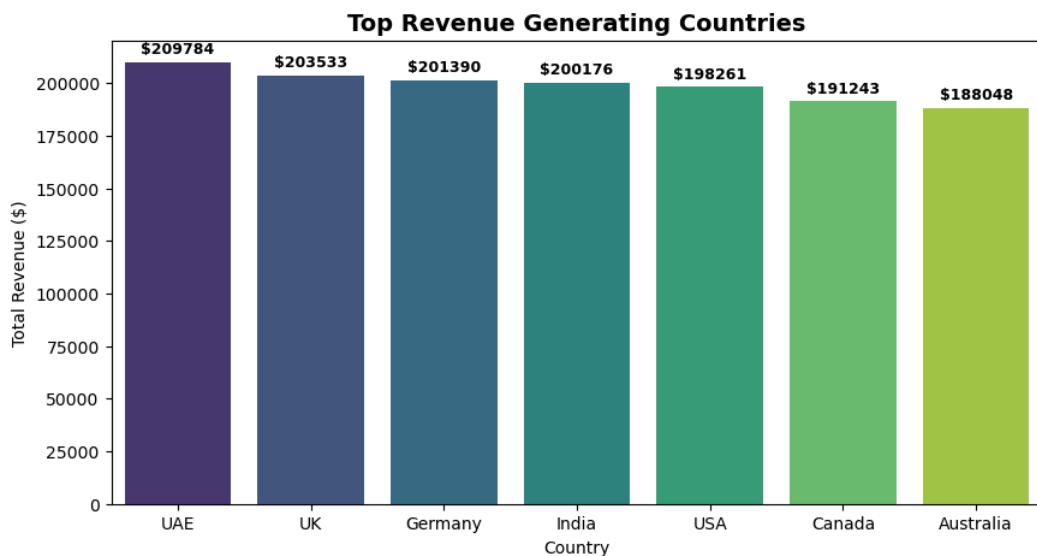
```
plt.ylabel("Total Revenue ($)")
plt.savefig('2_revenue_by_category.png', dpi=300, bbox_inches='tight')
plt.show()
```



Country Wise Revenue (Bar Chart)

```
In [32]: plt.figure(figsize=(10,5))
country_sales=df.groupby("Country")["Total Amount"].sum().sort_values(ascending=False).reset_index()

ax=sns.barplot(
    x="Country",
    y="Total Amount",
    hue="Country",
    data=country_sales,
    palette="viridis"
)
for i in ax.containers:
    ax.bar_label(
        i,
        fmt="%.0f",
        padding=3,
        fontweight="bold",
        fontsize=9
    )
plt.title(
    "Top Revenue Generating Countries",
    fontsize=14,
    fontweight="bold"
)
plt.xlabel("Country")
plt.ylabel("Total Revenue ($)")
plt.savefig('3_country_revenue.png', dpi=300, bbox_inches='tight')
plt.show()
```



Preferred Payment Methods (Pie Chart)

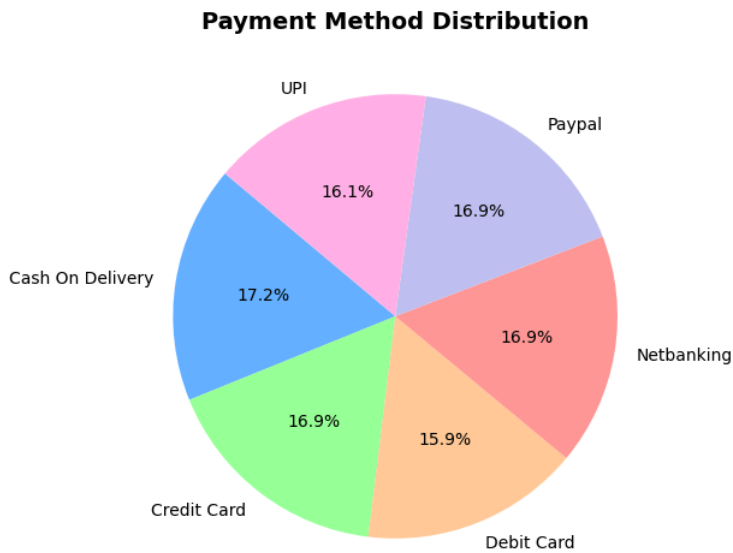
```
In [33]: plt.figure(figsize=(6,6))
payment_types=df.groupby("Payment Method")["Payment Method"].value_counts()

plt.pie(
    payment_types,
    labels=payment_types.index,
    autopct="%1.1f%%",
    colors=["#66b3ff", "#99ff99", "#ffcc99", "#ff9999", "#c2c2f0", "#ffb3e6"],
    startangle=140
)
plt.title(
    "Payment Method Distribution",
```

```

        fontsize=14,
        fontweight="bold"
    )
plt.savefig('4_payment_method_distribution.png', dpi=300, bbox_inches='tight')
plt.show()

```



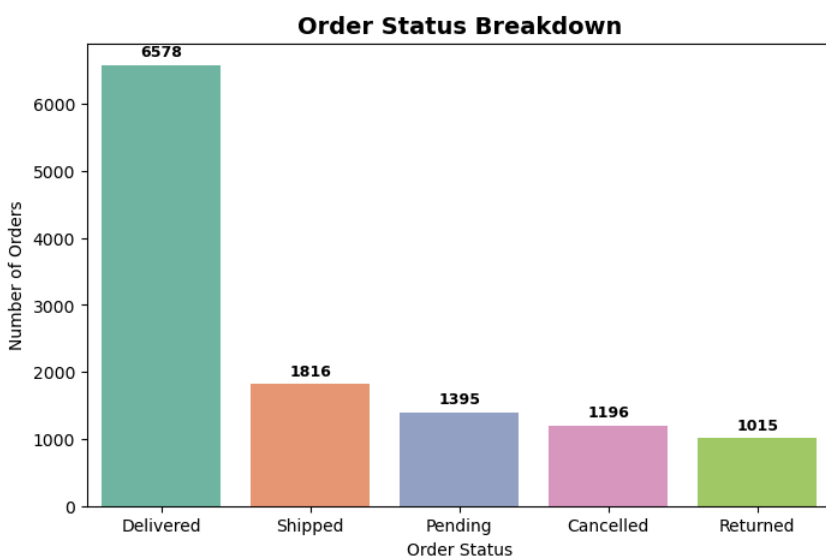
Order Status Breakdown (Count Plot)

```

In [34]: plt.figure(figsize=(8,5))
order_status=df.groupby("Order Status")["Order ID"].count().sort_values(ascending=False).reset_index()

ax=sns.barplot(
    x="Order Status",
    y="Order ID",
    data=order_status,
    hue="Order Status",
    palette="Set2"
)
for i in ax.containers:
    ax.bar_label(
        i,
        fmt="%0f",
        padding=3,
        fontweight="bold",
        fontsize=9
    )
plt.title(
    "Order Status Breakdown",
    fontsize=14,
    fontweight="bold"
)
plt.xlabel("Order Status")
plt.ylabel("Number of Orders")
plt.savefig('5_order_status_breakdown.png', dpi=300, bbox_inches='tight')
plt.show()

```



Top 10 Revenue Generating Cities

```

In [35]: plt.figure(figsize=(12,6))
top_cities=df.groupby("Shipping City")["Total Amount"].sum().sort_values(ascending=False).head(10).reset_index()

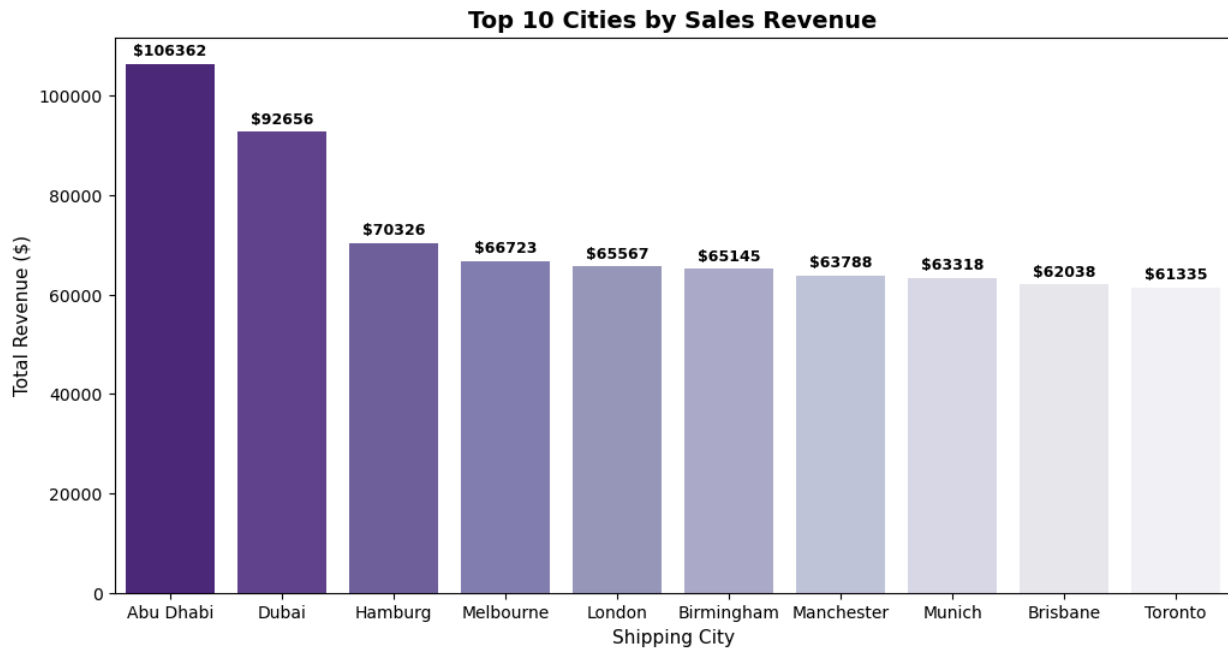
ax=sns.barplot(
    x="Shipping City",
    y="Total Amount",
    data=top_cities,
    hue="Shipping City",

```

```

palette="Purples_r",
legend=False
)
for i in ax.containers:
    ax.bar_label(
        i,
        fmt="%.0f",
        padding=3,
        fontweight="bold",
        fontsize=9
    )
plt.title(
    "Top 10 Cities by Sales Revenue",
    fontsize=14,
    fontweight="bold"
)
plt.xlabel(
    "Shipping City",
    fontsize=11
)
plt.ylabel(
    "Total Revenue ($)",
    fontsize=11
)
plt.savefig('6_top_cities_revenue.png', dpi=300, bbox_inches='tight')
plt.show()

```



Top 10 Selling Products

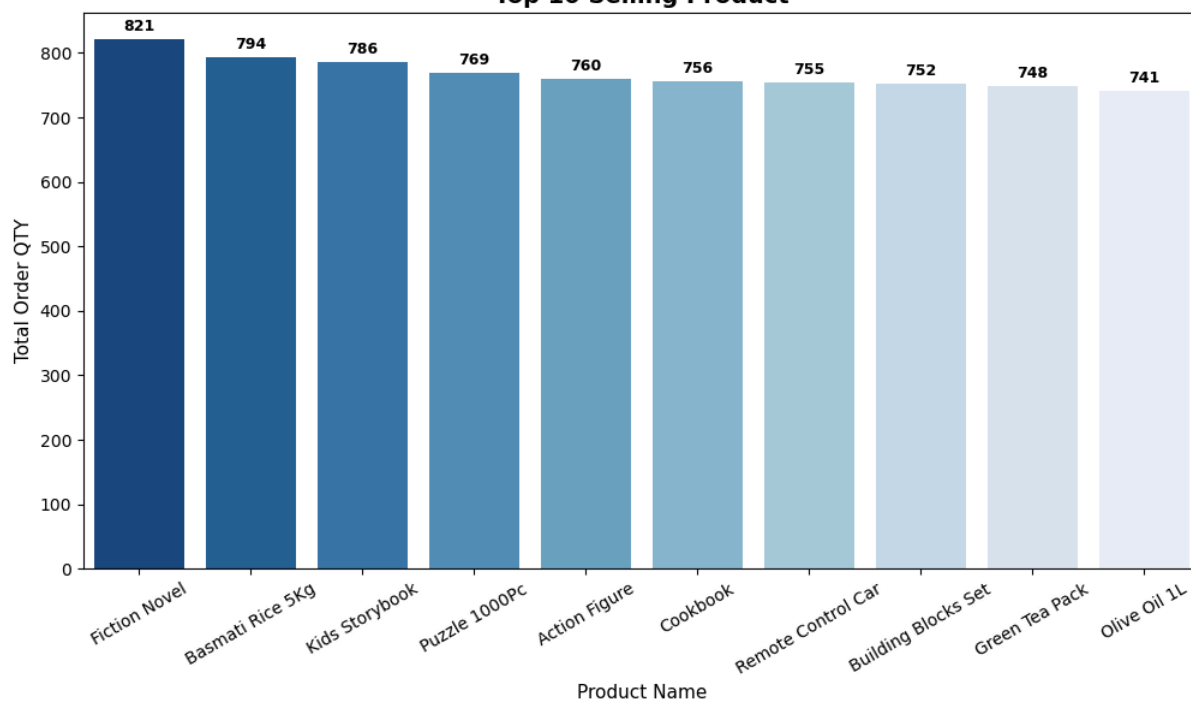
```

In [36]: plt.figure(figsize=(12,6))
top_product=df.groupby("Product Name")["Quantity"].sum().sort_values(ascending=False).head(10).reset_index()

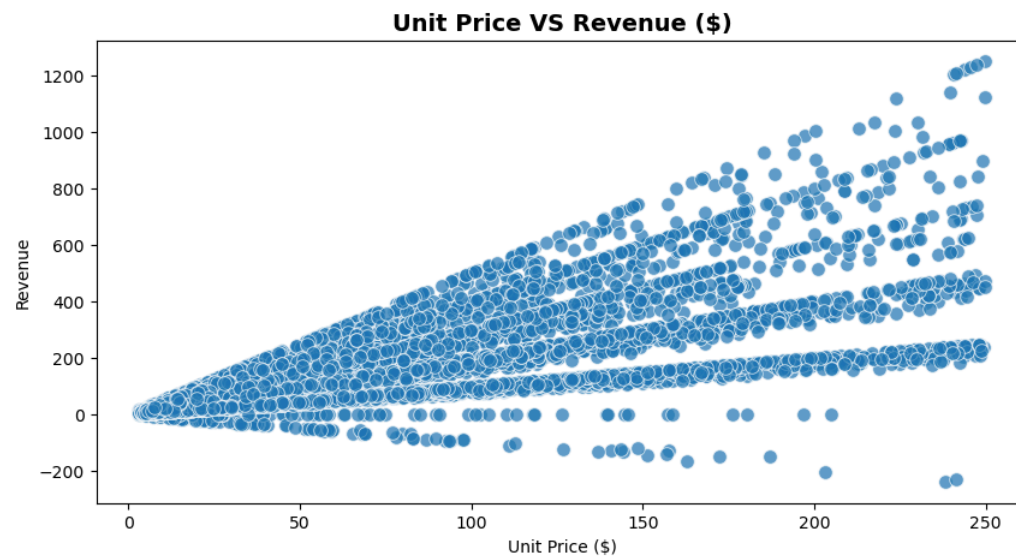
ax=sns.barplot(
    x="Product Name",
    y="Quantity",
    data=top_product,
    hue="Product Name",
    palette="Blues_r",
    legend=False,
)
for i in ax.containers:
    ax.bar_label(
        i,
        fmt="%.0f",
        padding=3,
        fontweight="bold",
        fontsize=9
    )
plt.title(
    "Top 10 Selling Product",
    fontsize=14,
    fontweight="bold"
)
plt.xlabel(
    "Product Name",
    fontsize=11
)
plt.ylabel(
    "Total Order QTY",
    fontsize=11
)
plt.xticks(rotation=30)
plt.savefig('7_top_selling_products.png', dpi=300, bbox_inches='tight')
plt.show()

```


Top 10 Selling Product

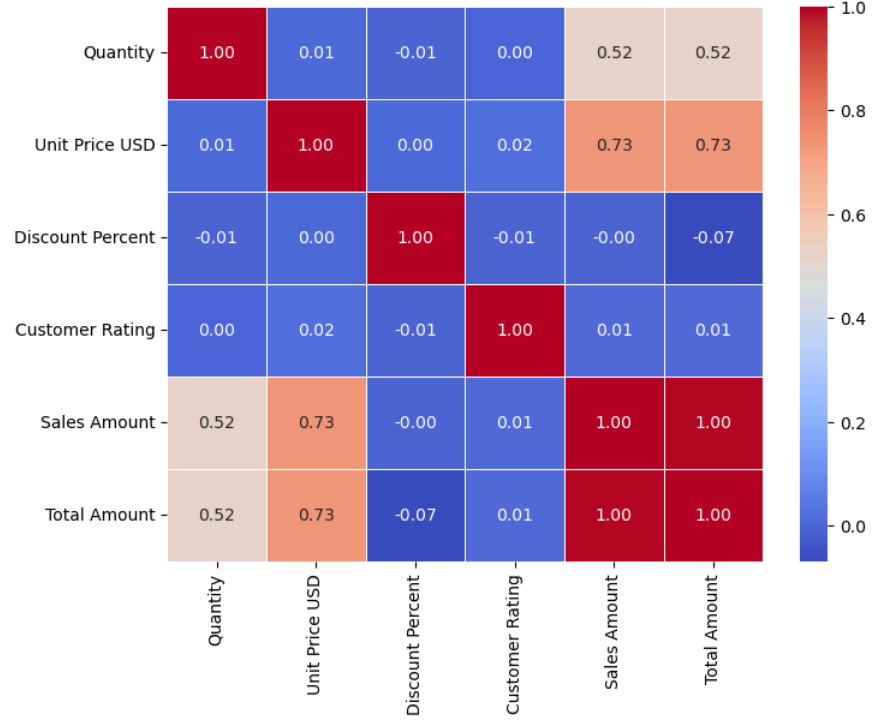


```
In [37]: plt.figure(figsize=(10,5))
sns.scatterplot(
    x="Unit Price USD",
    y="Total Amount",
    data=df,
    alpha=0.7,
    s=70,
    legend=False
)
plt.title(
    "Unit Price VS Revenue ($)",
    fontsize=14,
    fontweight="bold"
)
plt.xlabel("Unit Price ($)")
plt.ylabel("Revenue")
plt.savefig('8_unit_price_vs_revenue.png', dpi=300, bbox_inches='tight')
plt.show()
```



```
In [38]: plt.figure(figsize=(8,6))
num_cols=df.select_dtypes(include=["float64", "int64"]).columns
corr=df[num_cols].corr()
sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm",
    fmt='.2f',
    linewidths=0.5
)
plt.title(
    "Correlation Matrix of Numerical Features",
    fontsize=14,
    fontweight="bold"
)
plt.savefig('9_correlation_matrix.png', dpi=300, bbox_inches='tight')
plt.show()
```

Correlation Matrix of Numerical Features



```
In [39]: # To save Clean Data
df.to_csv("C:\\Users\\Dell\\Desktop\\Project 1\\Ecommerce retail\\ecommerce_retail_transactions_cleaned.csv", index=False)
```